CORE–13: Artificial Intelligence (Unit–2)

Problem Solving and Searching Techniques: In Artificial Intelligence, Search techniques are universal problem-solving methods. Rational agents or Problem-solving agents in AI mostly used these search strategies or algorithms to solve a specific problem and provide the best result. Problem-solving agents are the goal-based agents and use atomic representation.

Search Algorithm Terminologies:

- **Search**: Searching is a step by step procedure to solve a search-problem in a given search space. A **search problem can have three main factors**:
    a. **Search Space**: Search space represents a set of possible solutions, which a system may have.
    b. **Start State**: It is a state from where agent begins the search.
    c. **Goal test**: It is a function which observe the current state and returns whether the goal state is achieved or not.
- **Search tree**: A tree representation of search problem is called Search tree. The root of the search tree is the root node which is corresponding to the initial state.
- **Actions**: It gives the description of all the available actions to the agent.
- **Transition model**: A description of what each action do, can be represented as a transition model.
- **Path Cost**: It is a function which assigns a numeric cost to each path.
- **Solution**: It is an action sequence which leads from the start node to the goal node.
- **Optimal Solution**: If a solution has the lowest cost among all solutions.

Properties of Search Algorithms: Following are the four essential properties of search algorithms to compare the efficiency of these algorithms:

- **Completeness**: A search algorithm is said to be complete if it guarantees to return a solution if at least any solution exists for any random input.
- **Optimality**: If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution for is said to be an optimal solution.
- **Time Complexity**: Time complexity is a measure of time for an algorithm to complete its task.
- **Space Complexity**: It is the maximum storage space required at any point during the search, as the complexity of the problem.

Problem Characteristics:

- Is the problem decomposable into a set of (nearly) independent smaller or easier sub-problems?
- Can solution steps be ignored or at least undone if they prove unwise?
- Is the problem's universe predictable?
- Is a good solution to the problem obvious without comparison to all other possible solutions?
- Is the desired solution a state of the world or path to state?

- Is a large amount of knowledge absolutely required to solve the problem, or is knowledge important only to constrain the search?
- Can a computer that is simply given the problem, return the solution, will the solution of the problem require interaction between the computer and person?

Production Systems: Production system provides the structure of describing and performing the search process in AI. It consists of the following:

Components of Production System: The major components of Production System in Artificial Intelligence are:

- **Global Database:** The global database is the central data structure used by the production system in Artificial Intelligence.
- **Set of Production Rules:** The production rules operate on the global database. Each rule usually has a precondition that is either satisfied or not by the global database. If the precondition is satisfied, the rule is usually applied. The application of the rule changes the database.
- **A Control System:** The control system then chooses which applicable rule should be applied and ceases computation when a termination condition on the database is satisfied. If multiple rules are to fire at the same time, the control system resolves the conflicts.

Classes of Production System in Artificial Intelligence: There are four major classes of Production System in Artificial Intelligence:

- **Monotonic Production System**: It's a production system in which the application of a rule never prevents the later application of another rule that could have also been applied at the time the first rule was selected.
- **Partially Commutative Production System**: It's a type of production system in which the application of a sequence of rules transforms state X into state Y, then any permutation of those rules that is allowable also transforms state x into state Y.
- **Non-Monotonic Production Systems**: These are useful for solving ignorable problems. These systems are important from an implementation standpoint because they can be implemented without the ability to backtrack to previous states when it is discovered that an incorrect path was followed. This production system increases efficiency since it is not necessary to keep track of the changes made in the search process.
- Commutative Production Systems: These are useful for problems in which changes occur but can be reversed and in which the order of operation is not critical.

Advantages of Production system-

- Provides excellent tools for structuring AI programs
- The system is highly modular because individual rules can be added, removed or modified independently
- Separation of knowledge and Control-Recognises Act Cycle
- A natural mapping onto state-space research data or goal-driven
- The system uses pattern directed control which is more flexible than algorithmic control

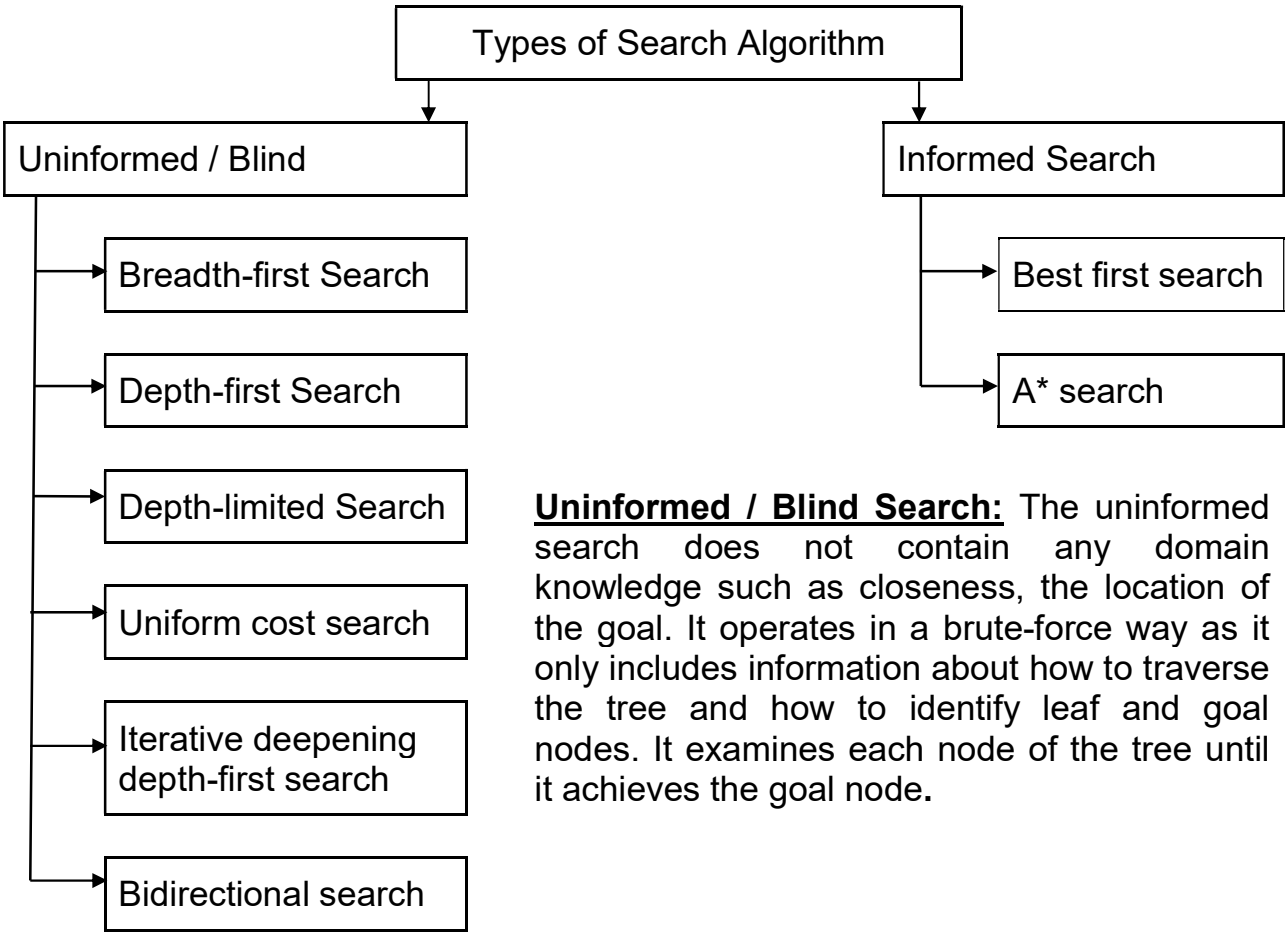
- Provides opportunities for heuristic control of the search
- A good way to model the state-driven nature of intelligent machines
- Quite helpful in a real-time environment and applications.

Disadvantages Production system:

- It is very difficult to analyze the flow of control within a production system
- There is an absence of learning due to a rule-based production system that does not store the result of the problem for future use.
- The rules in the production system should not have any type of conflict resolution as when a new rule is added to the database it should ensure that it does not have any conflict with any existing rule.

Control strategies: Control Strategy in Artificial Intelligence scenario is a technique or strategy that tells us about which rule has to be applied next while searching for the solution of a problem within problem space. A good Control strategy has three main characteristics:

- **Control Strategy should cause Motion:** Each rule or strategy applied should cause the motion because if there will be no motion than such control strategy will never lead to a solution.
- **Control strategy should be Systematic:** Though the strategy applied should create the motion but if do not follow some systematic strategy than we are likely to reach the same state number of times before reaching the solution which increases the number of steps.
- Finally, it must be **efficient** in order to find a good answer.



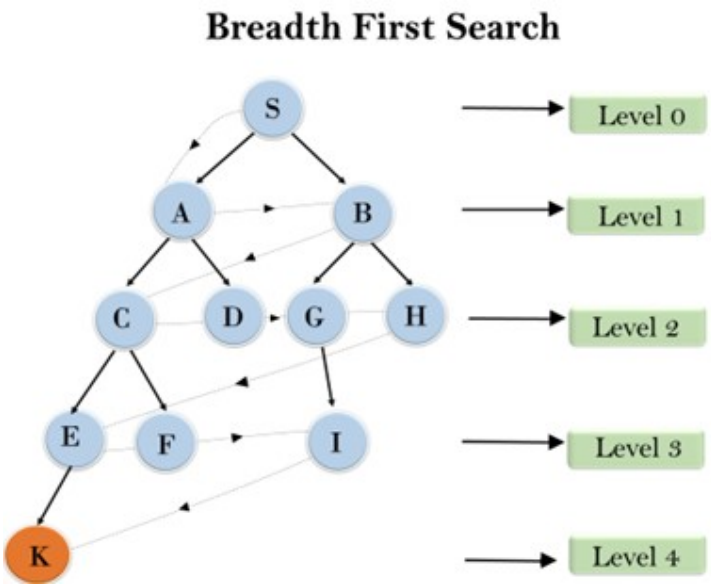
Uninformed / Blind Search: The uninformed search does not contain any domain knowledge such as closeness, the location of the goal. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes. It examines each node of the tree until it achieves the goal node.

Informed Search: Informed search algorithms use domain knowledge. In an informed search, problem information is available which can guide the search. Informed search strategies can find a solution more efficiently than an uninformed search strategy. Informed search is also called a **Heuristic search**. A heuristic is a way which might not always be guaranteed for best solutions but guaranteed to find a good solution in reasonable time. Informed search can solve much complex problem which could not be solved in another way.

Breadth-first Search:

- Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadth wise in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the root node of the tree and expands all successor nodes at the current level before moving to nodes of next level.
- Breadth-first search is implemented using FIFO **queue** data structure.

Example:
In this tree structure, we have shown the traversing of the tree using BFS algorithm from the root node S to goal node K. BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:
S---> A--->B--->C--->D--->G--->H--->E--->F--->I--->K



Space Complexity:
Space complexity of BFS algorithm is given by the Memory size which is **$O(b^d)$.**

Time Complexity: Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where d = depth of shallowest solution and b is the highest level of the tree (Problem space).
$T(b) = 1+b^2+b^3+.....+ b^d =$ **$O(b^d)$**

Completeness: BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.

Advantages:

- BFS will provide a solution if any solution exists.
- If there is more than one solution for a given problem, then BFS will provide the minimal solution which requires the least number of steps.

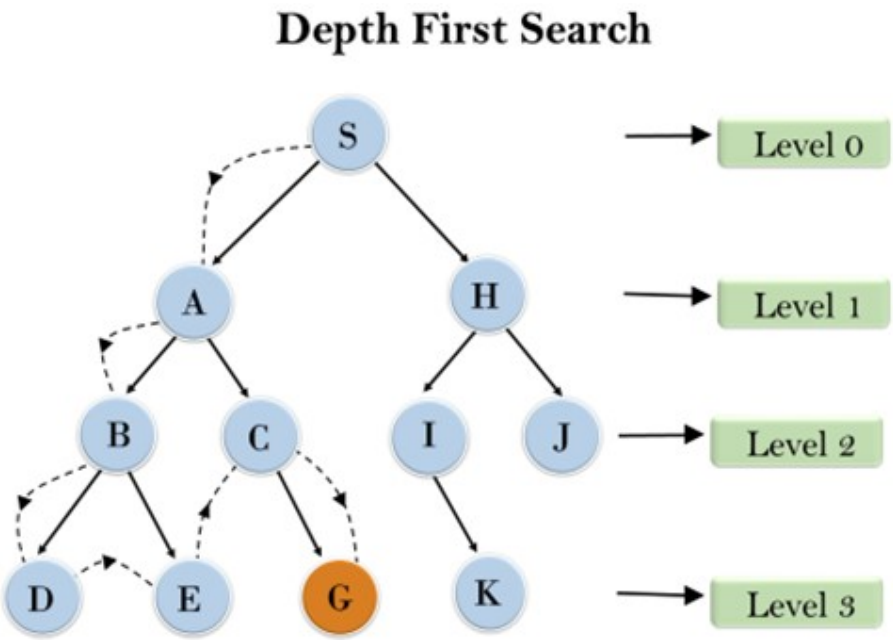
Disadvantages:

- It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
- BFS needs lots of time if the solution is far away from the root node.

Depth-first Search:

- Depth-first search is a recursive algorithm for traversing a tree or graph data structure.
- It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.
- DFS uses a **stack** data structure for its implementation.
- The process of the DFS algorithm is similar to the BFS algorithm.

Example: Depth-first search follow the order as: Root node--->Left node ----> right node. It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.



Completeness: DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by: $T(n)= 1+ b^2+ b^3 +.....+ b^d =$ **$O(b^d)$** Where, b = branching factor and d = maximum depth of any node.

Space Complexity: DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is **O(bd).**

Advantage:

- DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
- It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).

Disadvantage:

- There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
- DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

Heuristics Search Techniques: (Informed Search Algorithms):

uninformed search algorithms looks through search space for all possible solutions of the problem without having any additional knowledge about search space. But informed search algorithm contains an array of knowledge such as **how** far we are from the goal, **path cost**, **how to reach** to goal node, etc. This knowledge helps agents to explore less to the search space and find more efficiently the goal node. The informed search algorithm is more useful for large search space. Informed search algorithm uses the idea of heuristic. So it is also called **Heuristic search**.

Heuristics function: Heuristic is a function which is used in Informed Search, and it finds the most promising path. It takes the current state of the agent as its input and produces the estimation of how close is the agent from the goal. The heuristic method, however, might not always give the best solution, but it guarantees to find a good solution in reasonable time. It is represented by **h(n),** and it calculates the cost of an optimal path between the pair of states. The value of the heuristic function is always positive. Admissibility of the heuristic function is given as: **h(n) <= h*(n)**
Where h(n) is heuristic cost, and h*(n) is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.

Pure Heuristic Search: Pure heuristic search is the simplest form of heuristic search algorithms. It expands nodes based on their heuristic value h(n). It maintains two lists, OPEN and CLOSED list. In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded. In each iteration, the node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list. The algorithm continues until a goal state is found. The two popular informed search techniques are:

- Best First Search
- A* Search

1) Best First Search: In BFS and DFS, when we are at a node, we can consider any of the adjacent as next node. So both BFS and DFS blindly explore paths without considering any cost function. The idea of Best First Search is to use an evaluation function to decide which adjacent node is most promising and then explore. Best-first search algorithm always selects the path which appears best at that moment. We use a **priority queue** to store costs of nodes. So the implementation is a variation of BFS, we just need to change Queue to Priority Queue. In the best first search algorithm, we expand the node 'n' which is closest to the goal node and the **closest cost** is estimated by the **heuristic function** i.e.
f(n) = h(n), Were, h(n) = estimated cost from node 'n' to the goal.

Best First Search algorithm:

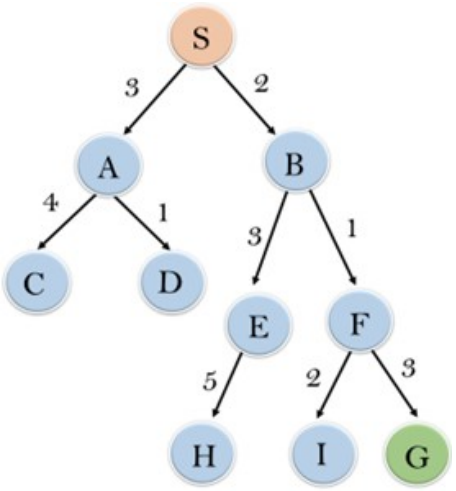
Step 1: Place the starting node into the OPEN list (a priority queue).
Step 2: If the OPEN list is empty, Stop and return failure.
Step 3: Remove the node n, from the OPEN list which has the lowest value of h(n), and places it in the CLOSED list.
Step 4: Expand the node n, and generate the successors of node n.

Step 5: Check each successor of node n, and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.

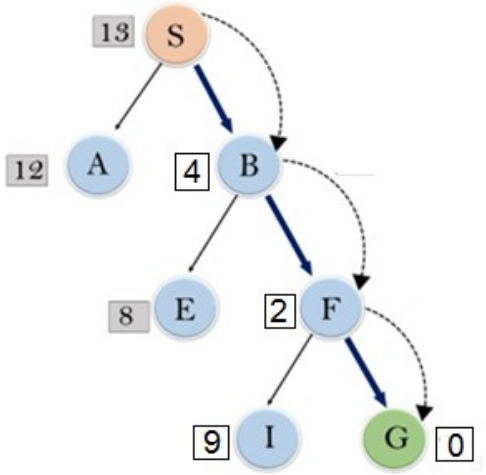
Step 6: For each successor node, algorithm checks for evaluation function f(n), and then check if the node has been in either OPEN or CLOSED list. If the node has not been in any of the list, then add it to the OPEN list.

Step 7: Return to Step 2.

Example: Consider the below search problem, and we will traverse it using Best First Search. At each iteration, each node is expanded using evaluation function f(n)=h(n) , which is given in the below table. In this search example, we are using two lists which are **OPEN** and **CLOSED** Lists. Following are the iteration for traversing.



Node	h(n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0



Solution: Following are the iteration for traversing.
Initialization: Open [A, B], Closed [S]
Iteration 1: Open [A], Closed [S, B]
Iteration 2: Open [E, F, A], Closed [S, B], Open [E, A], Closed [S, B, F]
Iteration 3: Open [I, G, E, A], Closed [S, B, F], Open [I, E, A], Closed [S, B, F, G]
Hence the final solution path will be: S----> B----->F----> G

Time Complexity: The worst case time complexity of best first search is $O(b^d)$, Where b is the branching factor and d is the maximum depth of the search space.

Space Complexity: The worst case space complexity of best first search is $O(b^d)$.

Complete: best-first search is also incomplete, even if the given state space is finite.

Optimal: Greedy best first search algorithm is not optimal.

Advantages:

- Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
- This algorithm is more efficient than BFS and DFS algorithms.

Disadvantages:

- It can behave as an unguided depth-first search in the worst case scenario.
- It can get stuck in a loop as DFS.
- This algorithm is not optimal.

2) A* Search Algorithm: A* Search algorithm is one of the best and popular technique used in path-finding and graph traversals. It uses the evacuation function

f(n) = g(n) + h(n) to reach the goal state, where

f(n) = estimated cost of the cheapest solution through the node 'n'.
g(n) = cost to reach the node 'n' from start state.
h(n) = cost to reach from node 'n' to goal state. This is often referred to as the heuristic function, which is nothing but a kind of **smart guess**.

Algorithm of A* search:

Step1: Place the starting node in the OPEN list.

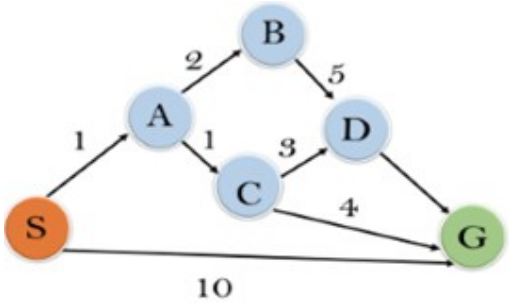
Step2: Check if the OPEN list is empty or not, if the list is empty then return failure and stops.

Step3: Select the node 'n' from the OPEN list which has the smallest value of evaluation function (g+h), if node 'n' is goal node then return success and stop, otherwise

Step4: Expand node 'n' and generate all of its successors, and put 'n' into the closed list. For each successor n', check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' and place into Open list.

Step5: Return to Step 2.

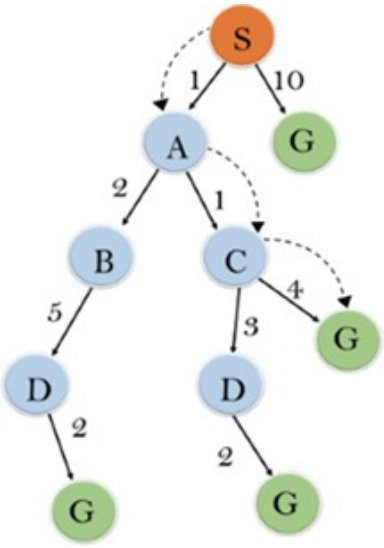
Example:
In this example, we will traverse the given graph using the A* algorithm. The heuristic value of all states is given in the table. So we will calculate f(n) of each state using the formula f(n) = g(n)+h(n).



State	H(n)
S	5
A	3
B	4
C	2
D	6
G	0

Here we will use OPEN and CLOSED list.

Solution:
Initialization: {(S, 5)}
Iteration1: {(**S→A, 4**), (S→G, 10)}
Iteration2: {(**S→A→C, 4**), (S→A→B, 7), (S→G, 10)}
Iteration3: {(**S→A→C→G, 6**), (S→A→C→D, 11),
(S→ A→B, 7), (S→G, 10)}
Iteration4 will give the final result, as **S→A→C→G** provides the optimal path with cost 6.



Time Complexity: The time complexity of A* search algorithm depends on heuristic function, and the number of nodes expanded is exponential to the depth of solution d. So the time complexity is **$O(b^d)$,** where b is the branching factor and d is the depth of expansion. The efficiency of A* algorithm depends on the quality of heuristic.

Space Complexity: The space complexity of A* search algorithm is **$O(b^d)$**

Complete: A* algorithm is complete as long as Branching factor is finite and Cost at every action is fixed.

Optimal: A* search algorithm is optimal if it follows below two conditions:

- **Admissible**: the first condition requires for optimality is that h(n) should be an admissible heuristic for A* tree search. An admissible heuristic is optimistic in nature.
- **Consistency**: Second required condition is consistency for only A* graph-search.

If the heuristic function is admissible, then A* tree search will always find the least cost path.

Hill climbing and its Variations:

- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem. It terminates when it reaches a peak value where no neighbour has a higher value.
- Hill climbing algorithm is a technique which is used for optimizing the mathematical problems. One of the widely discussed examples of Hill climbing algorithm is Travelling-salesman Problem in which we need to minimize the distance travelled by the salesman.
- It is also called greedy **local search** as it only looks to its good immediate neighbour state and not beyond that.
- A node of hill climbing algorithm has two components which are state and value.
- Hill Climbing is mostly used when a good heuristic is available.
- In this algorithm, we don't need to maintain and handle the search tree or graph as it only keeps a single current state.

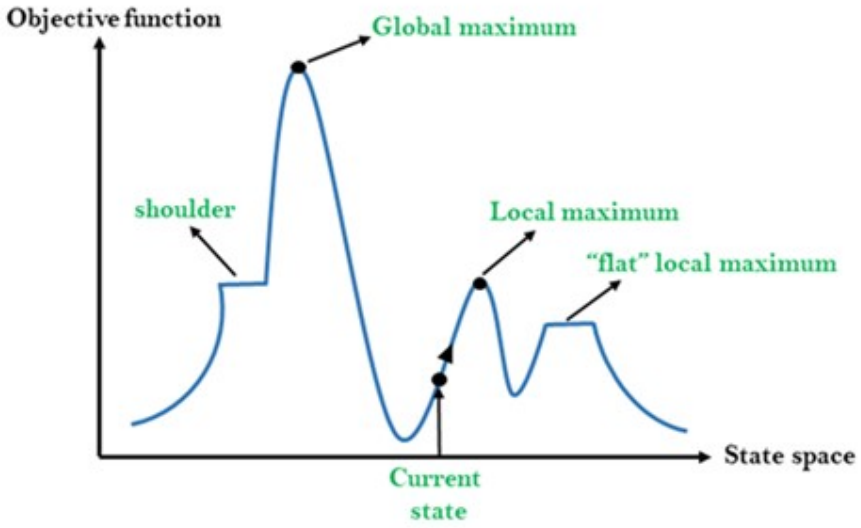
Features of Hill Climbing: Following are some of the main features of Hill Climbing Algorithm:

- **Generate and Test variant**: Hill Climbing is the variant of Generate and Test method. The Generate and Test method produce feedback which helps to decide which direction to move in the search space.
- **Greedy approach**: Hill-climbing algorithm search moves in the direction which optimizes the cost.
- **No backtracking**: It does not back-track the search space, as it does not remember the previous states.

State-space Diagram for Hill Climbing:

The state-space diagram is a graphical representation of the hill-climbing algorithm which is showing a graph between various states of algorithm and Objective function/Cost.

On Y-axis we have taken the function which can be an objective function or cost function, and state-space on the x-axis.



If the function on Y-axis is cost then, the goal of search is to find the global minimum and local minimum. If the function of Y-axis is Objective function, then the goal of the search is to find the global maximum and local maximum.

Different regions in the state space landscape:

Local Maximum: Local maximum is a state which is better than its neighbour states, but there is also another state which is higher than it.

Global Maximum: Global maximum is the best possible state of state space landscape. It has the highest value of objective function.

Current state: It is a state in a landscape diagram where an agent is currently present.

Flat local maximum: It is a flat space in the landscape where all the neighbour states of current states have the same value.

Shoulder: It is a plateau region which has an uphill edge.

Types of Hill Climbing Algorithm:

- Simple hill Climbing:
- Steepest-Ascent hill-climbing:
- Stochastic hill Climbing:

1. Simple Hill Climbing: Simple hill climbing is the simplest way to implement a hill climbing algorithm. It only evaluates the **neighbour node state** at a time and selects the first one which optimizes current cost and set it as a current state. It only checks it's one successor state, and if it finds better than the current state, then move else be in the same state. This algorithm has the following features:

- Less time consuming
- Less optimal solution and the solution is not guaranteed

Algorithm for Simple Hill Climbing:

Step 1: Evaluate the initial state, if it is goal state then return success and Stop.
Step 2: Loop Until a solution is found or there is no new operator left to apply.
Step 3: Select and apply an operator to the current state.
Step 4: Check new state:
If it is goal state, then return success and quit.
Else-if it is better than the current state then assign new state as a current state.
Else if not better than the current state, then return to step2.
Step 5: Exit.

2. Steepest-Ascent hill climbing: The steepest-Ascent algorithm is a variation of simple hill climbing algorithm. This algorithm **examines all the neighbouring nodes** of the current state and selects one neighbour node which is closest to the goal state. This algorithm consumes more time as it searches for multiple neighbours

Algorithm for Steepest-Ascent hill climbing:

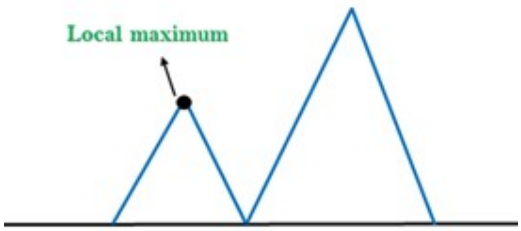
Step 1: Evaluate the initial state, if it is goal state then return success and stop, else make current state as initial state.
Step 2: Loop until a solution is found or the current state does not change.
a. Let SUCC be a state such that any successor of the current state will be better than it.
b. For each operator that applies to the current state:
i. Apply the new operator and generate a new state.
ii. Evaluate the new state.
iii. If it is goal state, then return it and quit, else compare it to the SUCC.
iv. If it is better than SUCC, then set new state as SUCC.
v. If the SUCC is better than the current state, then set current state to SUCC.
Step 5: Exit.

3. Stochastic hill climbing: Stochastic hill climbing **does not examine for its entire neighbour** before moving. Rather, this search algorithm **selects one neighbour node at random** and decides whether to choose it as a current state or examine another state.

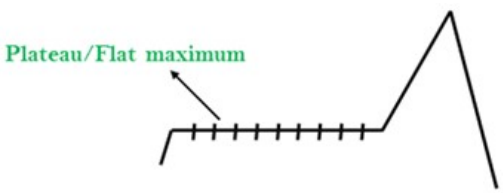
Problems in Hill Climbing Algorithm:

1. Local Maximum: A local maximum is a peak state in the state-space which is better than each of its neighbouring states, but there is another state also present which is higher than the local maximum.

Solution: Backtracking technique can be a solution of the local maximum in state space. Create a list of the promising path so that the algorithm can back-track the search space and explore other paths as well.



2. Plateau: A plateau is the flat area of the search space in which all the neighbour states of the current state contains the same value, because of this algorithm does not find any best direction to move. A hill-climbing search might be lost in the plateau area.



Solution: Randomly select a state which is far away from the current state so it is possible that the algorithm could find non-plateau region.

3. Ridges: A ridge is a special form of the local maximum. It has an area which is higher than its surrounding areas, but itself has a slope, and cannot be reached in a single move.
Solution: With the use of **bidirectional search**, or by moving in different directions, we can improve this problem.



Constraint Satisfaction Problem (CSP): A CSP consists of three things;

1. V = Finite set of **Variables** V_1 , V_2 , …, V_n
2. D = Nonempty **Domain** of possible values for each variable D_1, D_2, … D_n
3. C = Finite set of **Constraints** C_1 , C_2 , …, C_m such that each constraint C_i involves some subset of the variables and specifies the allowable combinations of values of that subset.

Popular Problems of CSP: The following problems are some of the popular problems that can be solved using CSP:

1. Crypt arithmetic (Coding alphabets to numbers.)
2. N-Queen (In an n-queen problem, n queens should be placed in a n x n matrix such that no queen shares the same row, column or diagonal.)
3. Map Colouring (Colouring different regions of map ensuring no adjacent regions have the same colour.) etc.

Solution Methods: We consider three solution methods for constraint satisfaction problems, *Generate-and-Test*, *Backtracking* and *Consistency Driven*.

Generate and Test: We generate one by one all possible complete variable assignments and for each we test if it satisfies all constraints. The corresponding program structure is very simple, just nested loops, one per variable. In the innermost loop we test each constraint.

Backtracking: We order the variables in some fashion, trying to place first the variables that are more highly constrained or with smaller ranges. This order has a great impact on the efficiency of solution algorithms and is examined elsewhere. We start assigning values to variables. We check constraint satisfaction at the earliest possible time and extend an assignment if the constraints involving the currently bound variables are satisfied.

Consistency Driven: Consistency Based Algorithms use information from the constraints to reduce the search space as early in the search as it is possible.

Backtracking Algorithm: The idea is to place queens one by one in different columns, starting from the leftmost column. When we place a queen in a column, we check for the conflict with already placed queens. In the current column, if we find a row for which there is no conflict, we mark this row and column as part of the solution. If we do not find such a row due to conflict then we backtrack and return false.

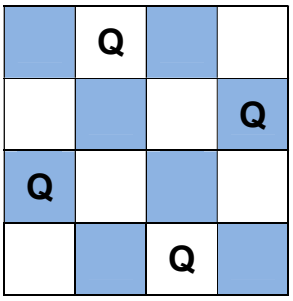
1) Start in the leftmost column
2) If all queens are placed then return true
3) Try all rows in the current column. Do the following for every tried row.
  a) If the queen can be placed safely in this row then mark this [row, column] as part of the solution and recursively check if placing queen here leads to a solution.
  b) If placing the queen in [row, column] leads to a solution then return true.
  c) If placing queen doesn't lead to a solution then unmark this [row, column] (Backtrack) and go to step (a) to try other rows.
4) If all rows have been tried and nothing worked, then return false to trigger backtracking.

Example-1: Crypt arithmetic (Each letter stands for a distinct digit. The aim is to find a substitution of digits for letters such that the resulting sum is arithmetically correct, with the added restriction that no leading zeroes are allowed.)

X3	X2	X1	
	T	W	O
+	T	W	O
F	O	U	R

Variables: F T U W R O, X1 X2 X3
Domain: {0,1,2,3,4,5,6,7,8,9}
Constraints:
Alldiff (F,T,U,W,R,O)
$O + O = R + 10 \cdot X1$
$X1 + W + W = U + 10 \cdot X2$
$X2 + T + T = O + 10 \cdot X3$
$X3 = F$

Exampmple-2 (N Queen Problem): The N Queen is the problem of placing N chess queens on an N×N chessboard so that no two queens attack each other (i.e. no two queens have the same row or same column or same diagonal). For example, here is a solution for 4 Queen Problem.



Example-3 (Map Colouring):
Here, Variables = {WA, NT, Q, NSW, V, SA, T}
Domain = {red, green, blue}
Constraints = {no adjacent regions have the same colour}

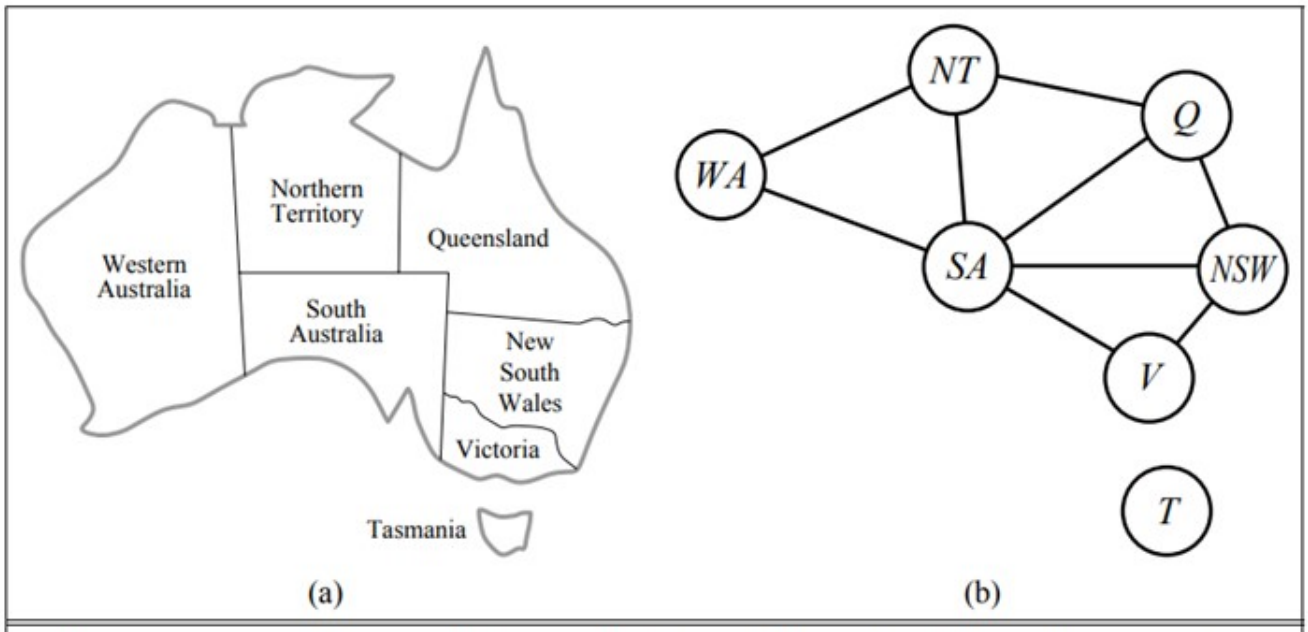


Figure (a) The principal states and territories of Australia. Coloring this map can be viewed as a constraint satisfaction problem. The goal is to assign colors to each region so that no neighboring regions have the same color. (b) The map-coloring problem represented as a constraint graph.

Solution: Here, the idea is we choose the region having maximum degree first and fill the colour satisfying the constraints.

	WA	NT	Q	NSW	V	SA	T
Initial domains	RGB	RGB	RGB	RGB	RGB	RGB	RGB
Take SA = red	GB	GB	GB	GB	GB	R	RGB
Take NT = green	B	G	B	GB	GB	R	RGB
Take NSW = green	B	G	B	G	B	R	RGB

Introduction to Game Playing: Game Playing is an important domain of artificial intelligence. Games don't require much knowledge; the only knowledge we need to provide is the rules, legal moves and the conditions of winning or losing the game. Both players try to win the game. So, both of them try to make the best move possible at each turn. Searching techniques like BFS (Breadth First Search) are not accurate for this as the branching factor is very high, so searching will take a lot of time. So, we need other search procedures. **The most common search technique in game playing is Min-Max search procedure. It is depth-first depth-limited search procedure.** It is used for games like chess and tic-tac-toe.

Min-Max Algorithm:

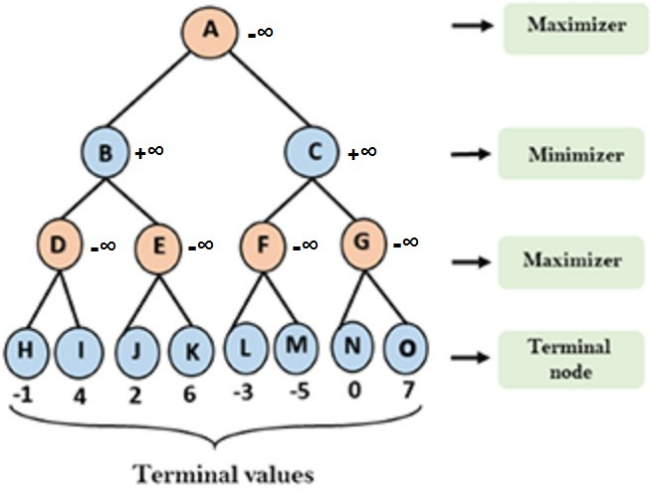
- Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game playing.
- It provides an optimal move for the player assuming that opponent is also playing optimally.

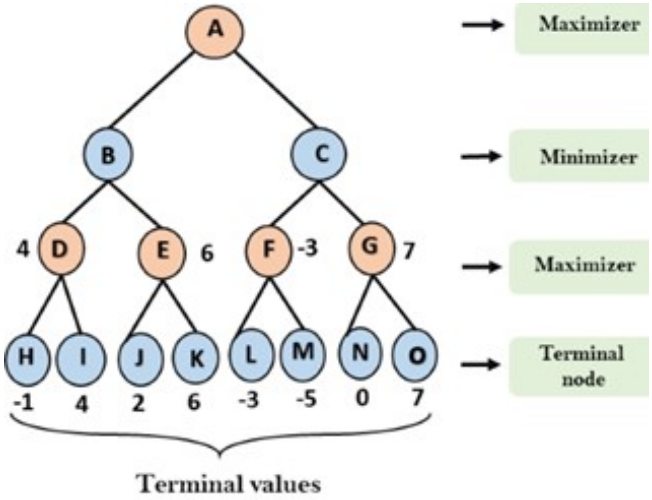
- It is mostly used for game playing in AI, such as Chess, tic-tac-toe and various tow-players game.
- In this algorithm **two players play the game**, **one is called MAX and other is called MIN.** Both the players fight assuming that the opponent player gets the minimum benefit while he gets the maximum benefit.
- The minimax algorithm performs a depth-first search algorithm for the exploration of the complete game tree. The algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.

Working of Min-Max Algorithm:

- The working of the minimax algorithm can be easily described using an example. Below we have taken an example of game-tree which is representing the two-player game.
- In this example, there are two players one is called Maximizer and other is called Minimizer.
- **Maximizer tries to get the Maximum possible score, and Minimizer tries that the opponent should get a minimum possible score.**
- This algorithm applies DFS, so in this game-tree, we have to go all the way through the leaves to reach the terminal nodes.
- At the terminal node, the terminal values are given so we will compare those values and backtrack the tree until the initial state occurs. Following are the main steps involved in solving the two-player game tree:

Step-1: In the first step, the algorithm generates the entire game-tree and applies the utility function to get the utility values for the terminal states. In the above tree diagram, let's take A is the initial state of the tree. Suppose maximizer takes first turn which has worst-case initial value= **-∞** infinity, and minimizer will take next turn which has worst-case initial value = **+∞**.



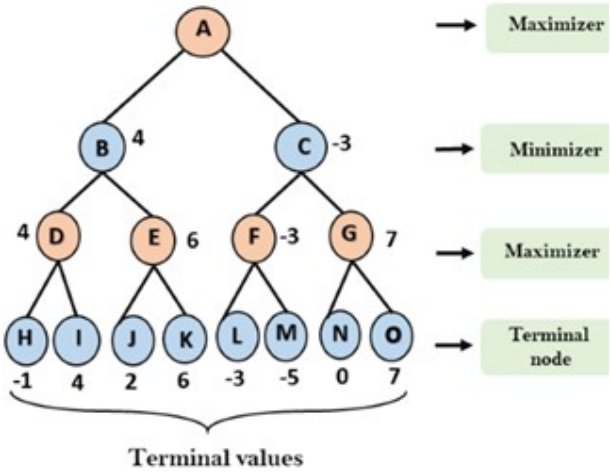


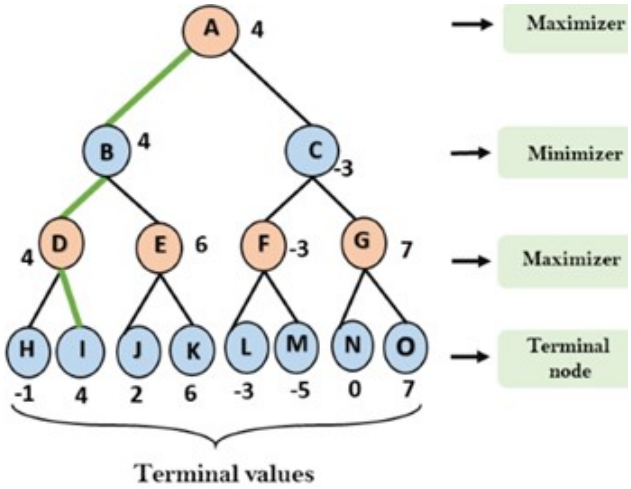
Step-2: Now, first we find the utilities value for the Maximizer, its initial value is -∞, so we will compare each value in terminal state with initial value of Maximizer and determines the higher nodes values. It will find the maximum among the all.

For node D, max(-1,4)= 4
For Node E, max(2, 6)= 6
For Node F, max(-3,-5) = -3
For node G, max(0, 7) = 7

Step-3: In the next step, it's a turn for minimizer, so it will compare all nodes value with +∞, and will find the 3rd layer node values.

For node B, min(4,6) = 4
For node C, min (-3, 7) = -3





Step-4: Now it's a turn for Maximizer, and it will again choose the maximum of all nodes value and find the maximum value for the root node. In this game tree, there are only 4 layers, hence we reach immediately to the root node, but in real games, there will be more than 4 layers.

- For node A max(4, -3)= 4

This is the complete workflow of the minimax two player game.

Properties of Mini-Max algorithm:

Complete- Min-Max algorithm is Complete. It will definitely find a solution (if exist), in the finite search tree.

Optimal- Min-Max algorithm is optimal if both opponents are playing optimally.

Time complexity- As it performs DFS for the game-tree, so the time complexity of Min-Max algorithm is **$O(b^d)$,** where b is branching factor of the game-tree, and d is the maximum depth of the tree.

Space Complexity- Space complexity of Mini-max algorithm is also similar to DFS which is **O(bd).**

Limitation of the minimax Algorithm: The main drawback of the minimax algorithm is that it gets really slow for complex games such as Chess. This type of games has a huge branching factor and the player has lots of choices to decide.

Alpha-Beta Pruning Algorithms:

- Alpha-beta pruning is a modified version of the minimax algorithm. It is an optimization technique for the minimax algorithm.
- As we have seen in the minimax search algorithm that the number of game states it has to examine are exponential in depth of the tree. But, **alpha-beta pruning** technique does not check each node of the game tree. It intelligently **cut or prunes** approximately half of the nodes of the game tree which are not checked. This involves two threshold parameter alpha and beta for future

expansion, so it is called **alpha-beta pruning**. It is also called as **Alpha-Beta Algorithm**.

- The two-parameter can be defined as:
  a. **Alpha**: The best (highest-value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is **-∞**.
  b. **Beta**: The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is **+∞**.
- The Alpha-beta pruning removes all the nodes which are not really affecting the final decision but making algorithm slow. Hence by pruning these nodes, it makes the algorithm fast.

The **main condition** for alpha-beta pruning is that if **α ≥ β** at any node then the **right branch of that node is pruned.**

Key points about alpha-beta pruning:

- The Max player will only update the value of alpha.
- The Min player will only update the value of beta.
- While backtracking the tree, the **node values will be passed to upper nodes** instead of values of alpha and beta.
- We will only **pass the alpha, beta values to the child nodes**.

Pseudo-code for Alpha-beta Pruning:

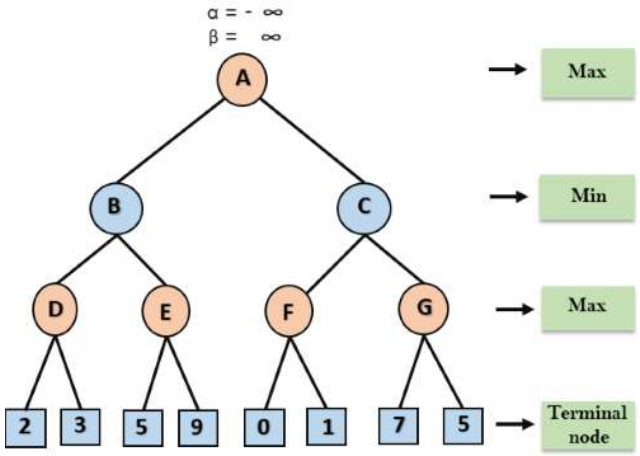
function minimax(node, depth, alpha, beta, maximizingPlayer)

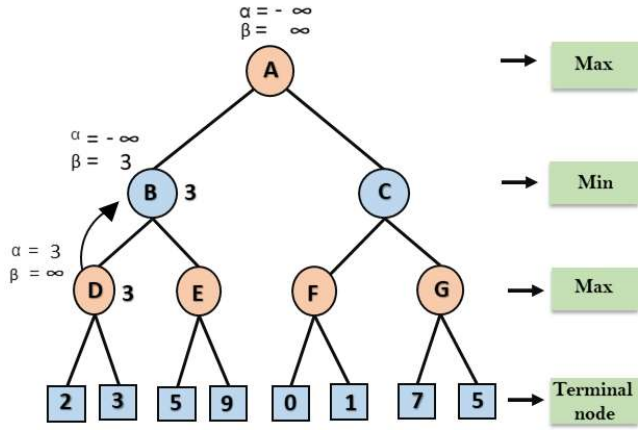
```
if depth = 0 or node is a terminal node then
        return static evaluation of node
if MaximizingPlayer then                // for Maximizer Player
        maxEva = -infinity
        for each child of node do
                eva = minimax(child, depth-1, alpha, beta, False)
                maxEva = max(maxEva, eva)
                alpha = max(alpha, maxEva)
                if beta<=alpha
                        break
         return maxEva

else                                    // for Minimizer player
        minEva= +infinity
        for each child of node do
                eva = minimax(child, depth-1, alpha, beta, true)
                minEva= min(minEva, eva)
                beta= min(beta, eva)
                if beta<=alpha
                        break
         return minEva
```

Example: Let's take an example of two-player search tree to understand the working of Alpha-beta pruning,

Step 1: At the first step the, Max player will start first move from node A where α = -∞ and β = +∞, these value of alpha and beta passed down to node B where again α = -∞ and β= +∞, and Node B passes the same value to its child D.



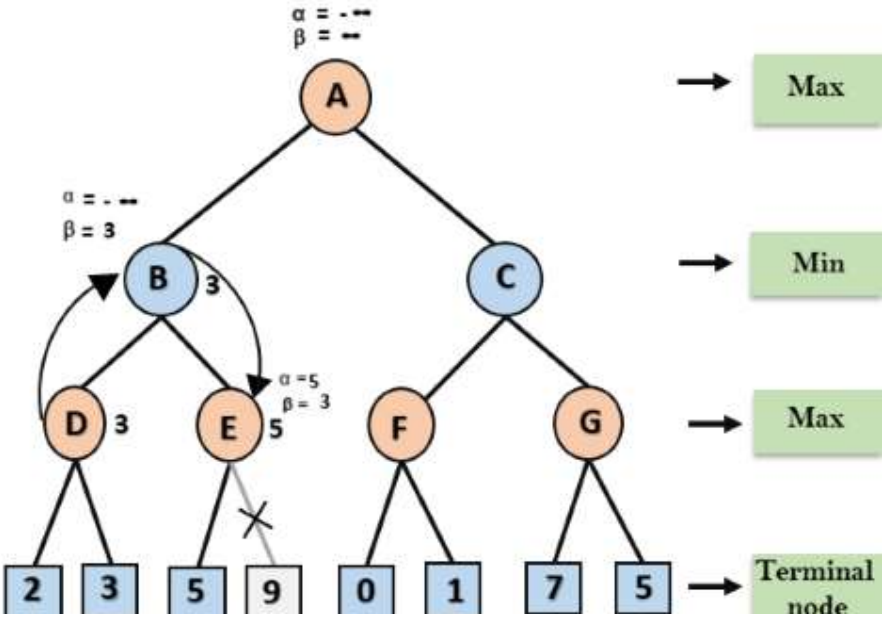


Step 2: At Node D, the value of α will be calculated as its turn for Max. The value of α is compared with firstly 2 and then 3, and max (2, 3) = 3 will be the value of α at node D and node value will also be 3.

Step 3: Now the algorithm backtracks to node B, where the value of β will change as this is a turn of Min, Now β = +∞, will compare with the available subsequent nodes value, i.e. min (∞, 3) = 3, hence at node B now α = -∞, and β = 3.

In the next step, algorithm traverse the next successor of Node B which is node E, and the values of α = -∞, and β = 3 will also be passed.

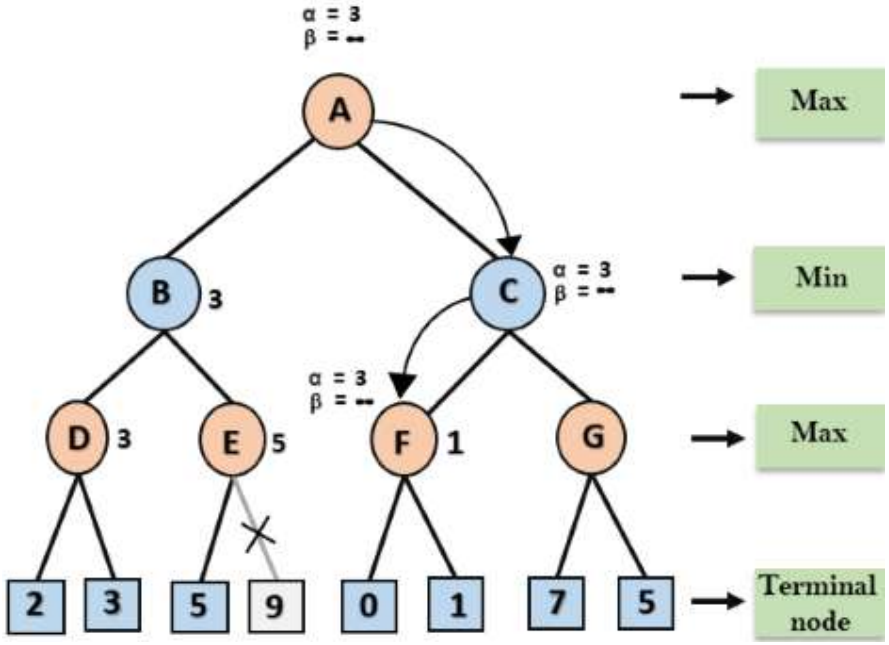
Step 4: At node E, Max will take its turn, and the value of alpha will change. The current value of alpha will be compared with 5, so max (-∞, 5) = 5, hence at node E, α = 5 and β = 3, where α ≥ β, so the right successor of E will be pruned, and algorithm will not traverse it, and the value at node E will be 5.



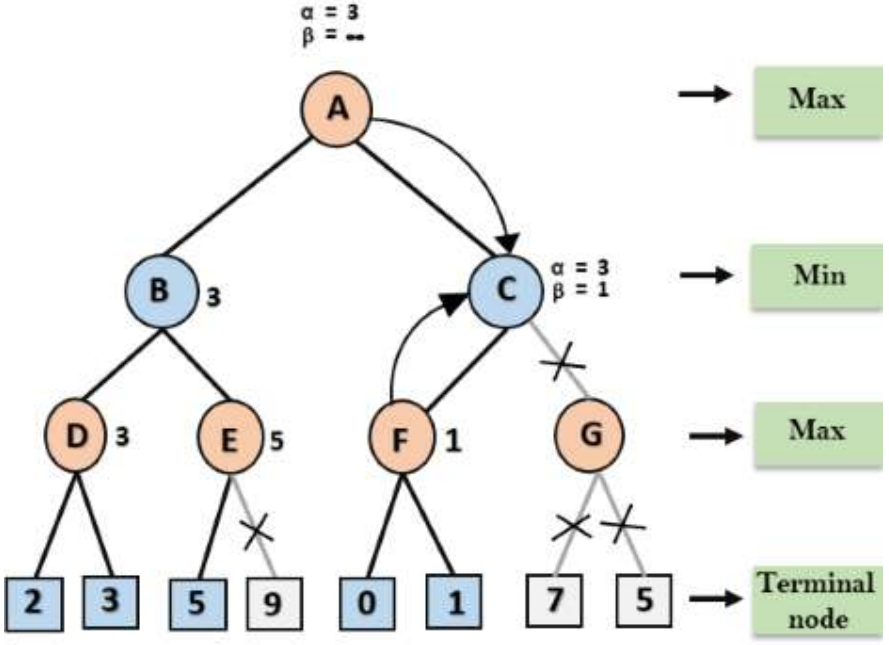
Step 5: At next step, algorithm again backtrack the tree, from node B to node A. At node A, the value of alpha will be changed the maximum available value is 3 as max (-∞, 3) = 3, and β = +∞, these two values now passes to right successor of A which is Node C.

At node C, α =3 and β = +∞, and the same values will be passed on to node F.

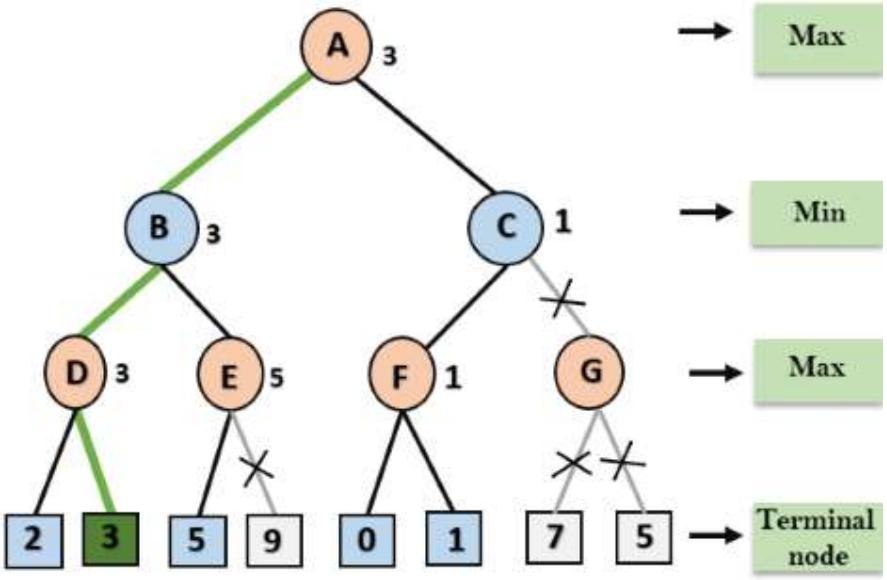
Step 6: At node F, again the value of α will be compared with left child which is 0, and max(3, 0) = 3, and then compared with right child which is 1, and max(3,1)= 3 still α remains 3, but the node value of F will become 1.



Step 7: Node F returns the node value 1 to node C, at C α= 3 and β= +∞, here the value of beta will be changed, it will compare with 1 so min (∞, 1) = 1. Now at C, α=3 and β= 1, and again it satisfies the condition α>=β, so the next child of C which is G will be pruned, and the algorithm will not compute the entire sub-tree G.



Step 8: C now returns the value of 1 to A here the best value for A is max (3, 1) = 3. Following is the final game tree which is the showing the nodes which are computed and nodes which has never computed. Hence the optimal value for the maximizer is 3 for this example.



Time Complexity: The **worst case** time complexity of alpha-beta pruning is **$O(b^d)$.** But the best and average case time complexity is **$O(b^{d/2})$.**